

Solution Requirements

Date	29 September 2026
Team ID	SWTID-2026-5841
Project Name	FitBuddy – AI Fitness Plan Generator
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Functional Requirements (FR) define what FitBuddy should do. Non-Functional Requirements (NFR) define how well it should do it.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Users identify themselves with a unique user ID (letters, digits, dash, underscore). Admin pages must require HTTP Basic login with credentials kept in environment variables.	High
2	Authorization levels	Two roles: User (generate a plan, submit feedback) and Admin (view all users, delete users).	High
3	External interfaces	Google Gemini API (google-genai SDK) for plan generation, plan revision and nutrition tip. SQLite database through SQLAlchemy.	High
4	Transactions processing	Each request is validated, sent to Gemini, and the user record (profile, original plan, updated plan, tip) is saved in one database session with rollback on failure.	High
5	Reporting	Admin dashboard listing all users with their details and plans; /health endpoint reporting service status.	Medium
6	Business rules, etc.	Age 13 to 100; weight above 0 and up to 500 kg; goal from five fixed goals; intensity low, medium or high; plan covers Day 1 to Day 7 with warm-up, main workout and cooldown; prompts forbid extreme exercise, starvation, dehydration and supplements; re-using a user ID resets that record.	High
7	Compliance to laws or regulations	Provides general wellness guidance only, not medical advice; advises consulting a professional for pain, injury or illness; collects only minimal personal data; API key and admin password are never committed to the repository.	Medium
8	Other	Friendly error pages for validation, unknown user and AI-service failures; Docker deployment.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Pages that do not call the AI respond quickly; plan generation depends on Gemini response time.	Home page under 2 seconds; plan or update under 15 seconds (typical)
2	Scalability	Stateless application; the database can be changed from SQLite to PostgreSQL through DATABASE_URL; several workers can run behind a proxy.	About 10 concurrent users on one instance; more with multiple workers
3	Security & Data Privacy	Secrets in environment variables and excluded by .gitignore; all inputs validated; feedback treated as data inside the prompt; admin credentials compared in constant time.	No secrets in the repository; invalid input rejected with 422
4	Reliability & Availability	Gemini calls retry up to 3 times with exponential backoff and a 60-second timeout; a failed nutrition tip does not block the plan.	Transient API errors recovered; clear error page otherwise
5	Usability & Accessibility	Single simple form, loading indicator while the AI works, responsive pages, plain-text plans that are easy to read.	Plan obtained in 3 steps: fill form, submit, read
6	Other	Maintainability and portability: modular code, automated tests with pytest, Docker image, README setup guide.	Project runs from the README in under 10 minutes